

Web System Technologies

IT0049



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Module 8

File Upload and Image Manipulation



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Objectives

- To be familiar with CodeIgniter File Upload Library.
- To set file permission and file type to be uploaded.
- To know the advantage of CI File Uploading's Security.
- To familiarize with the native Image Manipulation libraries.
- To manage image in term of Resizing, Thumbnail Creation, Cropping, Rotating, and Watermarking
- To learn how to downscale an image when uploaded.

File Upload

File Upload

CodeIgniter makes working with files uploaded through a form much simpler and more secure than using PHP's **\$_FILES** array directly. This extends the **File class** and thus gains all of the features of that class.

Note: This is not the same as the File Uploading class in CodeIgniter 3. This provides a raw interface to the uploaded files with a few small features.

File Upload Form Tutorial

Uploading a file involves the following general process:

- An upload form is displayed, allowing a user to select a file and upload it.
- When the form is submitted, the file is uploaded to the destination you specify.
- Along the way, the file is validated to make sure it is allowed to be uploaded based on the preferences you set.
- Once uploaded, the user will be shown a success message.

To demonstrate this process here is a brief tutorial. Afterward you'll find reference information.

Creating the Upload Form

Using a text editor, create a form called **upload_form.php**. In it, place this code and save it to your **app/Views** directory:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <title>Upload Form</title>
</head>
<body>

<?php foreach ($errors as $error): ?>
    <li><?= esc($error) ?></li>
<?php endforeach ?>

<?= form_open_multipart('upload/upload') ?>
    <input type="file" name="userfile" size="20">
    <br><br>
    <input type="submit" value="upload">
</form>

</body>
</html>
```



FEU ALABANG



FEU DILIMAN



FEU TECH

nology Driven by Innovation

The Success Page

Using a text editor, create a form called **upload_success.php**. In it, place this code and save it to your **app/Views** directory:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <title>Upload Form</title>
</head>
<body>

<h3>Your file was successfully uploaded!</h3>

<ul>
    <li>name: <?= esc($uploaded_fileinfo->getBasename()) ?></li>
    <li>size: <?= esc($uploaded_fileinfo->getSizeByUnit('kb')) ?> KB</li>
    <li>extension: <?= esc($uploaded_fileinfo->guessExtension()) ?></li>
</ul>

<p><?= anchor('upload', 'Upload Another File!') ?></p>

</body>
</html>
```



Controller

Using a text editor, create a controller called **Upload.php**. In it, place this code and save it to your **app/Controllers** directory:

```
<?php
namespace App\Controllers;

use CodeIgniter\Files\File;

class Upload extends BaseController
{
    protected $helpers = ['form'];

    public function index()
    {
        return view('upload_form', ['errors' => []]);
    }

    public function upload()
    {
        $validationRule = [
            'userfile' => [
                'label' => 'Image File',
                'rules' => [
                    'uploaded[userfile]',
                    'is_image[userfile]',
                    'mime_in[userfile,image/jpg,image/jpeg,image/gif,image/png,image/webp]',
                    'max_size[userfile,100]',
                    'max_dims[userfile,1024,768]',
                ],
            ],
        ];

        if (! $this->validateData([], $validationRule)) {
            $data = ['errors' => $this->validator->getErrors()];

            return view('upload_form', $data);
        }

        $img = $this->request->getFile('userfile');

        if (! $img->hasMoved()) {
            $filepath = WRITEPATH . 'uploads/' . $img->store();

            $data = ['uploaded_fileinfo' => new File($filepath)];

            return view('upload_success', $data);
        }

        $data = ['errors' => 'The file has already been moved.'];

        return view('upload_form', $data);
    }
}
```



The routes

Using a text editor, open **app/Config/Routes.php**. In it, add the following two routes:

```
<?php

// ...

/*
 * -----
 * Route Definitions
 * -----
 */

// We get a performance increase by specifying the default
// route since we don't have to scan directories.
$routes->get('/', 'Home::index');

$routes->get('upload', 'Upload::index');           // Add this line.
$routes->post('upload/upload', 'Upload::upload'); // Add this line.

// ...
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

The Upload Directory

The uploaded files are stored in the **writable/uploads/** directory.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Try It!

To try your form, visit your site using a URL similar to this one:

```
example.com/index.php/upload/
```

You should see an upload form. Try uploading an image file (either a **jpg**, **gif**, **png**, or **webp**). If the path in your controller is correct it should work.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Accessing Files

All Files

When you upload files they can be accessed natively in PHP through the \$_FILES superglobal. This array has some major shortcomings when working with multiple files uploaded at once, and has potential security flaws many developers are not aware of. CodeIgniter helps with both of these situations by standardizing your usage of files behind a common interface.

```
<?php  
  
$files = $this->request->getFiles();
```



Accessing Files

Of course, there are multiple ways to name the file input, and anything but the simplest can create strange results. The array returns in a manner that you would expect. With the simplest usage, a single file might be submitted like:

```
<input type="file" name="avatar">
```

Which would return a simple array like:

```
[  
    'avatar' => // UploadedFile instance,  
];
```

Accessing Files

Single File

If you just need to access a single file, you can use **getFile()** to retrieve the file instance directly.

With the simplest usage, a single file might be submitted like:

```
<input type="file" name="userfile">
```

Which would return a simple file instance like:

```
<?php  
  
$file = $this->request->getFile('userfile');
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Accessing Files

Multiple Files

```
<input type="file" name="images[]" multiple>
```

In controller:

```
<?php  
  
if ($imagefile = $this->request->getFiles()) {  
    foreach ($imagefile['images'] as $img) {  
        if ($img->isValid() && ! $img->hasMoved()) {  
            $newName = $img->getRandomName();  
            $img->move(WRITEPATH . 'uploads', $newName);  
        }  
    }  
}
```

where the **images** is a loop from the form field name.



Image Manipulation

Image Manipulation Class

Image Manipulation class lets you perform the following actions:

- Image Resizing
- Thumbnail Creation
- Image Cropping
- Image Rotating
- Image Watermarking

The following image libraries are supported: **GD/GD2**, and **ImageMagick**.

Initializing the Class

Like most other classes in CodeIgniter, the image class is initialized in your controller by calling the global function **service()**:

```
<?php  
  
$image = service('image');
```

You can pass the alias for the image library you wish to use into the global function **service()**:

```
<?php  
  
$image = service('image', 'imagick');
```

The available Handlers are as follows:

- gd - The GD/GD2 image library
- imagick - The ImageMagick library.

Note: If using the ImageMagick library, you must set the path to the library on your server in **app/Config/Images.php**.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Processing an Image

Regardless of the type of processing you would like to perform (resizing, cropping, rotation, or watermarking), the general process is identical. You will set some preferences corresponding to the action you intend to perform, then call one of the available processing functions.

```
<?php  
  
$image->withFile('/path/to/image/mypic.jpg')  
    ->fit(100, 100, 'center')  
    ->save('/path/to/image/mypic_thumb.jpg');
```

The above code tells the library to look for an image called **mypic.jpg** located in the **/path/to/image** folder, then create a new image from it that is 100 x 100 pixels, and save it to a new file **mypic_thumb.jpg**. Since it is using the `fit()` method, it will attempt to find the best portion of the image to crop based on the desired aspect ratio, and then crop and resize the result.

Processing an Image

An image can be processed through as many of the available methods before saving. The original image is left untouched, and a new image is used and passed through each method, applying the results on top of the previous results:

```
<?php  
  
$image->withFile('/path/to/image/mypic.jpg')  
    ->reorient()  
    ->rotate(90)  
    ->crop(100, 100, 0, 0)  
    ->save('/path/to/image/mypic_thumb.jpg');
```

This example would take the same image and first fix any mobile phone orientation issues, rotate the image by 90 degrees, and then crop the result into a 100 x 100 pixel image, starting at the top left corner. The result would be saved as the thumbnail.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Image Quality

save() can take an additional parameter **\$quality** to alter the resulting image quality. Values range from 0 to 100 with 90 being the framework default. This parameter only applies to JPEG and WebP images, will be ignored otherwise:

```
<?php

$image->withFile('/path/to/image/mypic.jpg')
    // processing methods
    ->save('/path/to/image/my_low_quality_pic.jpg', 10);
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Image Quality

If you are only interested in changing the image quality without doing any processing. You will need to include the image resource or you will end up with an exact copy:

```
<?php  
  
$image->withFile('/path/to/image/mypic.jpg')  
->withResource()  
->save('/path/to/image/my_low_quality_pic.jpg', 10);
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Processing Methods

There are seven available processing methods:

- `$image->crop()`
- `$image->convert()`
- `$image->fit()`
- `$image->flatten()`
- `$image->flip()`
- `$image->resize()`
- `$image->rotate()`
- `$image->text()`



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Processing Methods

These methods return the class instance so they can be chained together, as shown. If they fail they will throw a ***CodeIgniter\Images\ImageException*** that contains the error message. A good practice is to catch the exceptions, showing an error upon failure, like this:

```
<?php

$image = service('image');

try {
    $image->withFile('/path/to/image/mypic.jpg')
        ->fit(100, 100, 'center')
        ->save('/path/to/image/mypic_thumb.jpg');
} catch (\CodeIgniter\Images\Exceptions\ImageException $e) {
    echo $e->getMessage();
}
```



Cropping Images

Images can be cropped so that only a portion of the original image remains. This is often used when creating thumbnail images that should match a certain size/aspect ratio. This is handled with the **crop()** method:

```
crop(int $width = null, int $height = null, int $x = null, int $y = null, bool $maintainRatio = false, string $masterDim = 'auto')
```

- **\$width** is the desired width of the resulting image, in pixels.
- **\$height** is the desired height of the resulting image, in pixels.
- **\$x** is the number of pixels from the left side of the image to start cropping.
- **\$y** is the number of pixels from the top of the image to start cropping.
- **\$maintainRatio** will, if true, adjust the final dimensions as needed to maintain the image's original aspect ratio.
- **\$masterDim** specifies which dimension should be left untouched when \$maintainRatio is true. Values can be: 'width', 'height', or 'auto'.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Cropping Images

To take a 50 x 50 pixel square out of the center of an image, you would need to first calculate the appropriate x and y offset values:

```
<?php

$info = service('image', 'imagick')
    ->withFile('/path/to/image/mypic.jpg')
    ->getFile()
    ->getProperties(true);

$xOffset = ($info['width'] / 2) - 25;
$yOffset = ($info['height'] / 2) - 25;

service('image', 'imagick')
    ->withFile('/path/to/image/mypic.jpg')
    ->crop(50, 50, $xOffset, $yOffset)
    ->save('/path/to/new/image.jpg');
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Converting Images

The **convert()** method changes the library's internal indicator for the desired file format. This doesn't touch the actual image resource, but indicates to **save()** what format to use:

```
convert(int $imageType)
```

\$imageType is one of PHP's image type constants (see for example <https://www.php.net/manual/en/function.image-type-to-mime-type.php>):

```
<?php  
  
service('image')  
    ->withFile('/path/to/image/mypic.jpg')  
    ->convert(IMAGETYPE_PNG)  
    ->save('/path/to/new/image.png');
```

Fitting Images

The fit() method aims to help simplify cropping a portion of an image in a “smart” way, by doing the following steps:

- Determine the correct portion of the original image to crop in order to maintain the desired aspect ratio.
- Crop the original image.
- Resize to the final dimensions.

```
fit(int $width, int $height = null, string $position = 'center')
```

- \$width is the desired final width of the image.
- \$height is the desired final height of the image.
- \$position determines the portion of the image to crop out. Allowed positions: 'top-left', 'top', 'top-right', 'left', 'center', 'right', 'bottom-left', 'bottom', 'bottom-right'

This provides a much simpler way to crop that will always maintain the aspect ratio:

```
<?php  
  
service('image', 'imagick')  
    ->withFile('/path/to/image/mypic.jpg')  
    ->fit(100, 150, 'left')  
    ->save('/path/to/new/image.jpg');
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Flattening Images

The `flatten()` method aims to add a background color behind transparent images (PNG) and convert RGBA pixels to RGB pixels

- Specify a background color when converting from transparent images to jpgs.

```
flatten(int $red = 255, int $green = 255, int $blue = 255)
```

- `$red` is the red value of the background.
- `$green` is the green value of the background.
- `$blue` is the blue value of the background.

```
<?php

service('image', 'imagick')
    ->withFile('/path/to/image/mypic.png')
    ->flatten()
    ->save('/path/to/new/image.jpg');

service('image', 'imagick')
    ->withFile('/path/to/image/mypic.png')
    ->flatten(25, 25, 112)
    ->save('/path/to/new/image.jpg');
```



Flipping Images

Images can be flipped along either their horizontal or vertical axis:

```
flip(string $dir)
```

- `$dir` specifies the axis to flip along. Can be either `'vertical'` or `'horizontal'`.

```
<?php
```

```
service('image', 'imagick')  
->withFile('/path/to/image/mypic.jpg')  
->flip('horizontal')  
->save('/path/to/new/image.jpg');
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Resizing Images

Images can be resized to fit any dimension you require with the **resize()** method:

```
resize(int $width, int $height, bool $maintainRatio = false, string $masterDim = 'auto')
```

- **\$width** is the desired width of the new image in pixels
- **\$height** is the desired height of the new image in pixels
- **\$maintainRatio** determines whether the image is stretched to fit the new dimensions, or the original aspect ratio is maintained.
- **\$masterDim** specifies which axis should have its dimension honored when maintaining ratio. Either **'width'**, **'height'**.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Resizing Images

When resizing images you can choose to maintain the ratio of the original image, or stretch/squash the new image to fit the desired dimensions.

If **\$maintainRatio** is true, the dimension specified by **\$masterDim** will stay the same, while the other dimension will be altered to match the original image's aspect ratio.

```
<?php  
  
service('image', 'imagick')  
    ->withFile('/path/to/image/mypic.jpg')  
    ->resize(200, 100, true, 'height')  
    ->save('/path/to/new/image.jpg');
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Rotating Images

The **rotate()** method allows you to rotate an image in 90 degree increments:

```
rotate(float $angle)
```

- `$angle` is the number of degrees to rotate. One of `90`, `180`, `270`.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Adding a Text Watermark

You can overlay a text watermark onto the image very simply with the **text()** method. This is useful for placing copyright notices, photographer names, or simply marking the images as a preview so they won't be used in other people's final products.

```
text(string $text, array $options = [])
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Adding a Text Watermark

The first parameter is the string of text that you wish to display. The second parameter is an array of options that allow you to specify how the text should be displayed:

```
<?php
service('image', 'imagick')
->withFile('/path/to/image/mypic.jpg')
->text('Copyright 2017 My Photo Co', [
    'color'      => 'ffff',
    'opacity'    => 0.5,
    'withShadow' => true,
    'hAlign'     => 'center',
    'vAlign'     => 'bottom',
    'fontSize'   => 20,
])
->save('/path/to/new/image.jpg');
```

The possible options that are recognized are as follows:

- **color** Text Color (hex number), i.e., `'#ff0000'`
- **opacity** A number between `0` and `1` that represents the opacity of the text.
- **withShadow** Boolean value whether to display a shadow or not.
- **shadowColor** Color of the shadow (hex number)
- **shadowOffset** How many pixels to offset the shadow. Applies to both the vertical and horizontal values.
- **hAlign** Horizontal alignment: `'left'`, `'center'`, `'right'`
- **vAlign** Vertical alignment: `'top'`, `'middle'`, `'bottom'`
- **hOffset** Additional offset on the x axis, in pixels
- **vOffset** Additional offset on the y axis, in pixels
- **fontPath** The full server path to the TTF font you wish to use. System font will be used if none is given.
- **fontSize** The font size to use. When using the GD handler with the system font, valid values are between `1` to `5`.



